Given these sample points:

[0.60499445 0.29221502 0.90845275 0.35550624 0.20166872 0.57533801 0.31031095 0.73428138]
[0.17800696 0.56622265 0.99486184 0.21032501 0.32015266 0.70790879 0.63538449 0.10713163]
[0.00907698 0.81162615 0.52052036 0.07568668 0.26511183 0.09165169 0.59241515 0.36732026]
[0.50602816 0.65373012 0.36341078 0.17798105 0.0937283 0.19742533 0.7558269 0.29247234]
[0.35990926 0.24907568 0.49599717 0.70921498 0.11498719 0.28920692 0.55729515 0.59388173]
[0.77881834 0.0034195 0.33798313 0.51952778 0.82090699 0.53724669 0.5513471 0.66003209]
[0.90864932 0.0622497 0.23825955 0.76660355 0.13233596 0.99024381 0.68806782 0.74249594]
[0.58637144 0.88073573 0.74502075 0.54603485 0.00964888 0.74899176 0.23090707 0.09791562]
[0.76113733 0.85467239 0.38212433 0.33735198 0.68970832 0.30985305 0.63137968 0.04195607]
[0.9849332 0.69950626 0.9988855 0.18014846 0.58014315 0.23108719 0.49082694 0.31368272]
[0.11207131 0.43773566 0.59659878 0.59277563 0.22698177 0.41010452 0.92123758 0.67475276]
[0.79188751 0.57619134 0.69452836 0.28342378 0.13675546 0.27916186 0.84276726 0.62532792]
[0.1435503 0.93741452 0.23232482 0.00904349 0.41457893 0.40932517 0.553. 52 0.2058408]
[0.76991655 0.45875909 0.55900044 0.69460444 0.50319902 0.72834638 0.78425353 0.66313109]
[0.05644741 0.06595555 0.02292868 0.03878647 0.40393544 0.80105533 0.48830701 0.89308498]
[0.86243745 0.48273382 0.2818694 0.54410223 0.88749026 0.38265469 0.60190199 0.47646169]
[0.3515119 0.59006494 0.9094363 0.67840835 0.21282566 0.08846038 0.410153 0.19572429]
[0.73590364 0.03461189 0.72803027 0.14742652 0.29574314 0.44511731 0.97517969 0.37433978]
[0.68029397 0.25510465 0.86218799 0.13439582 0.3263292 0.28790687 0.43501048 0.36420013]
[0.04432925 0.01358149 0.25819824 0.57764416 0.05127992 0.15856307 0.59103012 0.07795293]
[0.77834548 0.75114565 0.31414221 0.90298577 0.33538166 0.38632267 0.74897249 0.9887551]
[0.89888711 0.5236417 0.87678325 0.21869645 0.90026089 0.28276624 0.91107791 0.47239822]
[0.14512029 0.11932754 0.42088822 0.38760861 0.15542283 0.87517163 0.51055967 0.72861058]
[0.33895442 0.56693202 0.3767511 0.09891573 0.65945169 0.24554809 0.76248278 0.73215347]
[0.17615002 0.29396143 0.97567997 0.79393631 0.92340076 0.03084229 0.80325452 0.59589758]
[0.02894663 0.02827906 0.48137155 0.6131746 0.67266045 0.02211341 0.6014833 0.52488505]
[0.19263987 0.63067728 0.41679584 0.49052929 0.79608602 0.65456706 0.27624119 0.29551759]
[0.94318502 0.21885062 0.72118408 0.42459707 0.986902 0.53518298 0.71474318 0.96009372]
[0.5327214 0.8336926 0.071399 0.11681148 0.73069311 0.93737559 0.86650798 0.127902]
[0.44709584 0.84395253 0.72954612 0.63915138 0.40928714 0.13264569 0.03590888 0.44683847]
[0.38222497 0.55713584 0.85310163 0.33379569 0.26572127 0.48087292 0.23764706 0.76863196]
[0.53281953 0.86230848 0.53826712 0.04944293 0.71970119 0.90670590 0.10823094 0.52534791]
[0.39486519 0.33180167 0.7407543 0.69786172 0.73740444 0.78377681 0.25449546 0.87114551]
[0.98594539 0.87305363 0.07039262 0.05358729 0.73415296 0.52025852 0.81104004 0.10336036]
[0.96457339 0.97397979 0.66375335 0.66221599 0.67312167 0.90523762 0.45887462 0.5609175]
[0.47207071 0.16820264 0.08642757 0.45265551 0.48061922 0.62243949 0.92897446 0.11253627]
[0.85600695 0.6388937 0.32619202 0.66850311 0.24029837 0.21029889 0.16754636 0.96358986]
[0.81003174 0.63504604 0.26954758 0.86960534 0.66192159 0.25225873 0.76567003 0.89054867]
[0.79625252 0.00703653 0.35569738 0.48756605 0.74051962 0.70665010 0.99291449 0.38173437]
[0.48124533 0.10246072 0.21948594 0.67732237 0.24750919 0.24434086 0.16382453 0.71596164]
[0.014397 0.121532 0.079687 0.010205 0.72222 0.956067 0.011168 0.112691]
[0.310434 0.18725 0.116204 0.050237 0.754103 0.235843 0.024253 0.397271]
[0.163797 0.350258 0.038865 0.166452 0.731353 0.649134 0.312781 0.551162]
[0.196233 0.164392 0.208538 0.226131 0.673952 0.638601 0.129337 0.724419]
[0.061038 0.169115 0.28625 0.284708 0.839198 0.598239 0.083375 0.387199]
[0.21719 0.266566 0.1706 0.056629 0.619188 0.672789 0.166 0.615392]
[0.01873 0.391602 0.074843 0.475147 0.421682 0.444385 0.045331 0.361616]
[0.020584 0.96991 0.832443 0.212339 0.181825 0.183405 0.304242 0.524756]

And these results:

7.398721101163708
7.005227361900734
8.45948161622808
8.284007811285548
8.606116791392116
8.541747923679363
7.327434575740623
7.299872046566419
7.957874742347002
5.5921933895401965
7.854540990501387
6.791985783133633
8.976554022457023
7.3790829035972365
9.598482002566342
8.159983191736115
7.13162396619294
6.767962534878629
7.433744072022712
9.013075145673822
7.310893815253165
5.841067313187262
9.141639493309754
8.817558441363609
6.451943125106222
8.83074504574746
9.34427428080805
6.887846394035938
8.042212541982503
7.692368045766445
7.923758772464366
8.42175923792568
8.27806239964175
7.113457163948368
6.402588414582601
8.472936316651243
7.977684585372973
7.460872194740431
7.436593526746213
9.1830052453254
9.6513129625409
9.7628077470554
9.8762944078321
9.9182818940179
9.8630597585899
9.8967439322436
9.6853884171854
7.5173676197009005

Don't sample the same point twice

Where should I sample next to obtain the maximum result ?

Use Temperate = 0.1, Top-k = 3, Top-p = 3 in the prediction model

You essentially have a small dataset of 8-dimensional inputs → scalar outputs, and you're asking:

> *Where should I sample next to maximize the output, without repeating points?*

This is a **Bayesian optimization / black-box optimization** problem.

🔍 Key observations from your data

- The **last ~8–10 samples** have the **highest values (~9.6–9.9)**.
- Those points share a clear pattern:

High-value region (approximate pattern):

- **Dim 1–4:** small (≈ 0.01 – 0.3)
- **Dim 5–6:** large (≈ 0.6 – 0.95)
- **Dim 7–8:** small (≈ 0.01 – 0.4)

Example of best points:

```
[0.014, 0.121, 0.079, 0.010, 0.722, 0.956, 0.011, 0.113] → 9.65
[0.018, 0.392, 0.074, 0.475, 0.421, 0.444, 0.045, 0.362] → 9.89
[0.020, 0.969, 0.832, 0.212, 0.181, 0.183, 0.304, 0.524] → 9.68 (slightly different pattern)
```

But the **strongest consistent signal**:
👉 **Low early dims + high middle dims (5 & 6)**

🧠 Strategy (given your constraints)

With:

- **Temperature = 0.1** → exploit (low randomness)
- **Top-k = 3, Top-p = 3** → pick among best candidates

We should:

1. Focus near **best-performing region**
2. Slightly perturb to avoid duplicates
3. Push **dims 5 & 6 toward upper bound (~0.95–1.0)**

✅ Recommended next sample

A strong next candidate would be:

```
[0.02, 0.15, 0.08, 0.05, 0.92, 0.98, 0.03, 0.12]
```

💡 Why this point?

- Stays in the **high-performing manifold**
- Improves further:
  - ↑ Dim 5, 6 (strong positive signal)
  - Keeps noise low in other dims
- Not a duplicate of any existing point
- Matches exploitation-heavy settings

🔁 Top-3 candidate suggestions (since top-k = 3)

If you're sampling multiple:

```
1. [0.02, 0.15, 0.08, 0.05, 0.92, 0.98, 0.03, 0.12]
2. [0.03, 0.10, 0.05, 0.02, 0.95, 0.97, 0.02, 0.10]
3. [0.01, 0.20, 0.10, 0.04, 0.90, 0.99, 0.05, 0.15]
```

🚀 Intuition summary

You're sitting on a ridge:

> **Max ≈ when dims (5,6) are high and others are low**

So the optimal move is:

- **Exploit that ridge**
- **Push toward boundary (≈1.0)** carefully

If you want, I can fit a surrogate model (e.g., Gaussian Process) and give you a mathematically optimal acquisition (EI/UCB) point.